bechtle

Technischer Workshop für Entwickler

Entwicklung mit Microsoft Foundry.

Modelle und Agents mit dem offiziellen JavaScript-SDK

Marcel Brodbeck
Bechtle Stuttgart

2,5 Stunden

bechtle

Ablauf.

Zeit	Block	Format
0:00 – 0:10	Einstieg, Zielbild, eure Use Cases	Diskussion
0:10 – 0:25	Foundry-Landkarte und Begriffsmodell	Slides
0:25 – 0:40	Modelle und Responses API	Slides + Demo
0:40 – 0:55	Lab 1 — Setup und erster Modellaufruf	Hands-on
0:55 – 1:05	Pause	
1:05 – 1:20	Agent Service: Agents, Conversations, Tools	Slides
1:20 – 1:45	Lab 2 — Agent anlegen und ansprechen	Hands-on
1:45 – 2:00	Lab 3 — Tools und MCP	Hands-on
2:00 – 2:15	Betrieb: Auth, Tracing, Evals, Kosten	Slides + Portal
2:15 – 2:30	Architektur am eigenen Use Case, Q&A	Offen

bechtle

Was ihr heute mitnehmt.

Ein laufender Agent im eigenen Projekt

Kein Slideware-Beispiel: am Ende steht Code, der gegen euren Tenant läuft und den ihr mitnehmt.

Die Architektur hinter dem SDK

Warum es zwei Clients gibt, wann welcher greift und wo die Grenze zwischen Foundry und OpenAI verläuft.

Eine Bewertungsgrundlage

Genug Überblick, um eure eigenen Use Cases einzuordnen: was trägt heute, was noch nicht.

bechtle

Kurze Runde: wo steht ihr?

01

Wer arbeitet heute schon mit LLM-APIs — egal bei welchem Anbieter?

02

Wer hat schon etwas in Foundry gebaut oder wenigstens das Portal gesehen?

03

Welcher konkrete Use Case bringt euch heute hierher?

bechtle

Azure AI Foundry heißt jetzt *Microsoft Foundry*.

Was ihr im Netz findet

Azure AI Studio · Azure AI Foundry · Foundry classic

Drei Namen, zwei Portalgenerationen, ein SDK-Major-Bruch. Ältere Blogposts, Forenantworten und Copilot-Vorschläge sitzen fast alle auf dem alten Stand.

Was wir heute nutzen

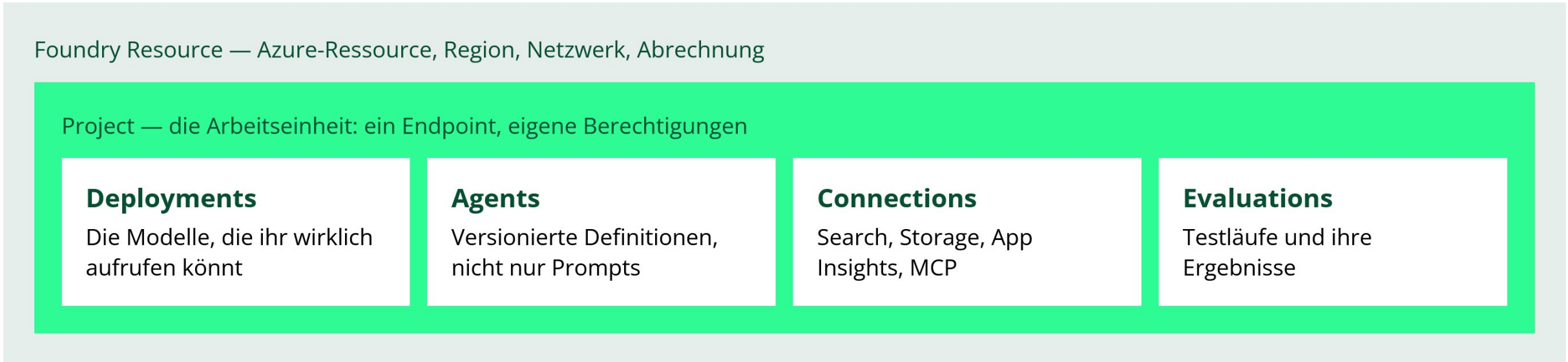
Microsoft Foundry · Projects (new) · SDK 2.x

Neues Portal, neues Ressourcenmodell, neue API-Fläche. Alles in diesem Workshop bezieht sich ausschließlich darauf.

Praktische Konsequenz: beim Suchen „Foundry“ statt „Azure AI Foundry“ verwenden und in der Doku auf den Versionshinweis achten. Wer das nicht tut, baut heute Nachmittag gegen eine API, die es so nicht mehr gibt.

bechtle

Das Ressourcenmodell.



Faustregel für die Projektstruktur: ein Project pro Team und Umgebung, nicht pro Use Case. Die Trennlinie verläuft dort, wo auch die Berechtigungen verlaufen sollen.

bechtle

Ein Endpoint statt fünf.

```
https://<resource>.services.ai.azure.com/api/projects/<project>
```

`/api/projects/…`

Foundry-eigene API

Agents verwalten, Connections lesen, Deployments auflisten, Tracing aktivieren.

`…/openai`

OpenAI-kompatibel

Responses, Conversations, Evals, Fine-Tuning und die Plattform-Tools.

`…/anthropic`

Messages API

Claude-Modelle in Foundry laufen über eine eigene Route, nicht über /openai.

Für die Anwendungskonfiguration heißt das: eine Umgebungsvariable statt einer Endpoint-Matrix. Custom Subdomain im Tenant? Dann steht sie anstelle des Ressourcennamens.

bechtle

Der Modellkatalog.

Azure OpenAI

Der Default für die meisten Projekte. Beste Abdeckung bei Tools und Features.

Foundry Direct

Von Azure verkaufte Katalogmodelle, über denselben Endpoint ansprechbar.

Claude in Foundry

Eigene Route, eigenes SDK, sonst dasselbe Berechtigungsmodell.

Open Weights

Für Spezialfälle und lokale Szenarien über Foundry Local.

Die Auswahl trifft man nicht nach Benchmark, sondern nach vier Fragen

Ist das Modell in eurer Region verfügbar? Passt es zur Vorgabe für die Datenresidenz? Unterstützt es die Tools, die ihr braucht? Und rechtfertigt der Qualitätsunterschied den Preisunterschied zum kleineren Geschwistermodell?

bechtle

Drei Wege ins selbe Backend.

Weg	Paket	Stärke	Was fehlt
Foundry SDK	@azure/ai-projects	Agents, Evals, Foundry-Tools, ein Endpoint	Etwas mehr Abstraktion als nötig für reine Modellaufrufe
OpenAI SDK direkt	openai gegen /openai/v1	Volle OpenAI-Fläche, beste Latenz, Embeddings	Keine Agents, keine Evaluations, keine Foundry-Tools
Anthropic SDK	@anthropic-ai/foundry-sdk	Claude-Modelle mit der Messages API	Eigene Route, nicht Teil der Agent-Welt

Wir bleiben heute durchgehend beim Foundry SDK. Wenn euer Use Case reines Prompt-in-Text-out ist und ihr keine Agents braucht, ist der direkte OpenAI-Weg die schlankere Wahl — das ist eine legitime Architekturentscheidung, kein Rückschritt.

bechtle

Zwei Clients, eine Anwendung.

Projekt-Client

Für alles, wofür OpenAI kein Gegenstück hat: Agents verwalten, Connections lesen, Projekteigenschaften, Tracing.

```
const project = new AIProjectClient(
  PROJECT_ENDPOINT,
  new DefaultAzureCredential()
);
```

OpenAI-kompatibler Client

Für alles mit OpenAI-Semantik: Responses, Conversations, Agent-Läufe, Evaluations, Fine-Tuning.

```
const openai =
  project.getOpenAIClient();
```

Merksatz: Projekt-Client für Setup und Konfiguration, OpenAI-Client für das Ausführen. Die meisten Anwendungen nutzen beide.

bechtle

Versionen: wo es heute knallt.

Stolperstein	Was zu tun ist
SDK 1.x gegen 2.x	2.x spricht die Projects-API, 1.x das alte Foundry classic. Der Code ist nicht kompatibel.
Node.js-Mindestversion	Seit 2.3.0 ist Node 22 Pflicht. Lockfile, CI-Runtime und eigene HTTP-Clients vorher prüfen.
Threads und Runs	Das Assistants-Muster ist weg. Heute: Agent-Version, Conversation, Response.
Evaluations	Seit 2.4.0 sind das Long-Running-Operations. Wer das Ergebnis sofort liest, bekommt nichts.
Copilot-Vorschläge	Trainingsdaten sind hier durchweg älter als die API. Gegen die aktuelle Doku prüfen, nicht gegen das Gefühl.

bechtle

Warum Responses und nicht Chat Completions.

Chat Completions

Ein Array von Nachrichten, das ihr selbst verwaltet

Der Zustand liegt komplett bei euch. Tools, Bilder und Reasoning-Modelle sind nachträglich angebaut. Funktioniert weiterhin, wächst aber nicht mit.

Responses API

Ein Lauf mit Ein- und Ausgabe-Items

Tools, Multimodalität und strukturierte Ausgaben sind Teil des Modells statt Anbau. Der Zustand kann optional serverseitig liegen. Grundlage für alles, was Agents tun.

Für Neuentwicklung gilt: Responses ist der Default. Bestehende Chat-Completions-Integrationen müsst ihr nicht anfassen, aber jede neue Funktion landet zuerst hier.

bechtle

Anatomie eines Aufrufs.

```
// npm i @azure/ai-projects @azure/identity dotenv
import { DefaultAzureCredential } from "@azure/identity";
import { AIProjectClient } from "@azure/ai-projects";

const project = new AIProjectClient(
  process.env.PROJECT_ENDPOINT,
  new DefaultAzureCredential()
);

const openai = project.getOpenAIClient();

const response = await openai.responses.create({
  model: "[Modell-Deployment]",
  input: "Fasse diesen Störungsbericht zusammen."
});

console.log(response.output_text);
```

Kein API-Key

Lokal greift euer az-login, in Azure die Managed Identity.

model = Deployment-Name

Nicht der Modellname aus dem Katalog. Häufigste Fehlerquelle im Lab.

output_text

Bequemer Shortcut. Darunter liegen einzelne Output-Items, die ihr auswerten könnt.

bechtle

Streaming.

```
const stream = await openai.responses.create({
  model: MODEL,
  input: prompt,
  stream: true
});

for await (const event of stream) {
  if (event.type === "response.output_text.delta") {
    process.stdout.write(event.delta);
  }
}
```

Streamen

Überall dort, wo ein Mensch auf die Antwort wartet: Chat-Oberflächen, Copilot-artige Assistenten, lange Zusammenfassungen.

Nicht streamen

Bei Batch-Verarbeitung, Klassifikation und allem, was in eine Queue schreibt. Dort kostet Streaming nur Komplexität.

Achtung bei Logic Apps, Functions und Power Automate: die meisten dieser Hosts verarbeiten Server-Sent-Events nicht sinnvoll.

bechtle

Strukturierte Ausgaben.

```
const response = await openai.responses.create({
  model: MODEL,
  input: ticketText,
  text: {
    format: {
      type: "json_schema",
      name: "ticket",
      strict: true,
      schema: ticketSchema
    }
  }
});
```

Schema statt Bitte

„Antworte bitte als JSON“ im Prompt ist eine Hoffnung. Ein Schema ist eine Zusage.

Der eigentliche Hebel

Erst damit wird ein Modell zu einer Komponente, die man in eine bestehende Automatisierung hängen kann — ohne Parser-Bastelei dahinter.

bechtle

Eine Ausnahme, die *jedes* RAG-Projekt trifft.

Embeddings werden über den Projekt-Endpoint nicht geroutet. Dafür braucht ihr den Azure-OpenAI-Endpoint direkt.

Symptom

Der Chat-Teil eurer RAG-Anwendung läuft, die Indexierung wirft einen Fehler, den niemand erwartet hat.

Konsequenz

In einer RAG-Architektur führt ihr zwei Endpoints und damit zwei Konfigurationspfade. Das gehört von Anfang an ins Design.

bechtle

Drei Aufrufe im Terminal.

1

Ein einfacher Aufruf, Antwort am Stück

2

Derselbe Aufruf gestreamt

3

Ein Ticket in ein JSON-Schema gegossen

bechtle

Lab 1 — Setup und erster Aufruf.

01

Repo klonen, Pakete ziehen

```
npm install
az login
```

02

.env füllen, dann prüfen

Projekt-Endpoint und Deployment-Name aus dem Portal übernehmen.

```
npm run doctor
```

03

Erster Aufruf

Dann als Erweiterung: streamen und ein JSON-Schema anhängen.

github.com/Mbeck89/Foundry_Development_Workshop · Ordner lab-01 · Die Lösung liegt in solution.js

bechtle

Pause.

10 Minuten

Wer sein Lab noch nicht zum Laufen gebracht hat: bleibt kurz da, wir schauen gemeinsam drauf.

bechtle

Was „Agent“ hier bedeutet.

Modell

Welches Deployment den Lauf ausführt

Instructions

Rolle, Grenzen, Tonfall — als Teil der Definition, nicht als Prompt im Code

Tools

Was der Agent darf: suchen, rechnen, MCP-Server aufrufen

Deklarativ, nicht prozedural

Der Agent ist ein Objekt im Projekt, das Portal und Code gleichermaßen sehen. Kein Klassenkonstrukt in eurer Anwendung.

Abgrenzung

Frameworks wie LangGraph oder das Agent Framework orchestrieren in eurem Prozess. Der Agent Service ist der verwaltete Gegenpart — beides lässt sich kombinieren.

bechtle

Einen Prompt Agent anlegen.

```
const project = new AIProjectClient(
  PROJECT_ENDPOINT,
  new DefaultAzureCredential()
);

const agent = await project.agents.createVersion(
  AGENT_NAME,
  {
    kind: "prompt",
    model: "[Modell-Deployment]",
    instructions: "Du beantwortest Fragen zum …"
  }
);

console.log(agent.name, agent.version);
```

createVersion, nicht create

Der Name ist stabil, jede Änderung erzeugt eine neue Version.

Projekt-Client

Verwalten läuft über project.agents — ausgeführt wird gleich über den OpenAI-Client.

Im Portal sichtbar

Derselbe Agent taucht sofort in der Oberfläche auf. Ein Objekt, zwei Zugänge.

bechtle

Versionierung ist der eigentliche Gewinn.

Ein Agent wird über Name *und* Version referenziert. Damit wird aus einer Prompt-Änderung ein nachvollziehbares Release statt eines Deployments mit Bauchgefühl.

Staging

Test zeigt auf die neue Version, Produktion bleibt auf der alten — ohne zweiten Agenten.

A/B

Zwei Versionen parallel laufen lassen und über Tracing vergleichen.

Rollback

Eine Versionsnummer zurückdrehen statt eine Prompt-Datei aus der Git-Historie fischen.

bechtle

Conversation speichert. Response führt aus.

```
const openai = project.getOpenAIClient();

// nur ein Kontextspeicher — hier passiert noch nichts
const conversation = await openai.conversations.create();

// erst das hier lässt das Modell laufen
const response = await openai.responses.create(
  { conversation: conversation.id,
    input: "Wie melde ich einen Ausfall?" },
  { body: { agent_reference: {
      name: agent.name, type: "agent_reference" } } }
);

console.log(response.output_text);
```

Häufigster Denkfehler

„Ich habe eine Conversation erzeugt, bekomme aber keine Antwort.“ Eine Conversation ruft das Modell nicht auf.

Ohne Conversation

Geht auch: einzelne Läufe ohne Historie. Die Conversation ist optional, kein Muss.

bechtle

Was ein Agent benutzen darf.

File Search
Dokumente durchsuchen ohne eigene Vektor-Pipeline

Code Interpreter
Rechnen, Dateien verarbeiten, Diagramme erzeugen

Web Search
Aktuelles außerhalb der Trainingsdaten

Memory
Wissen über Sitzungen hinweg behalten

SharePoint
Auf bestehende M365-Inhalte zugreifen

Fabric IQ
Fragen gegen das eigene Data Estate

MCP-Server
Eure eigenen Systeme über ein offenes Protokoll

Eigene Functions
Der Klassiker: ihr führt aus, das Modell entscheidet wann

Die ersten sieben laufen beim Dienst, das achte bei euch. Diese Trennlinie entscheidet über Netzwerkfreigaben, Datenabfluss und Betriebsaufwand — nicht die Funktionalität.

bechtle

Eigene Function oder MCP-Server?

Kriterium	Function Calling	MCP-Server
Wiederverwendung	An eure Anwendung gebunden	Einmal gebaut, von jedem Client nutzbar
Betrieb	Läuft in eurem Prozess mit	Eigener Dienst mit eigenem Lebenszyklus
Auth	Erbt den Kontext eurer Anwendung	Braucht ein eigenes Konzept — der schwierige Teil
Passt für	Eine Handvoll Aufrufe in einem Projekt	Fachliche Fähigkeiten, die mehrere Teams brauchen

Pragmatische Reihenfolge: mit Function Calling anfangen, und erst dann auf MCP wechseln, wenn dieselbe Fähigkeit im zweiten Projekt gebraucht wird.

bechtle

Portal und Code sind dasselbe Objekt.

1

Agent im Portal anlegen und kurz testen

2

Dieselbe Definition per SDK schreiben

3

Portal neu laden: die neue Version ist da

bechtle

Lab 2 — Agent und Conversation.

01

Agent anlegen

Eigene Instructions für einen Use Case aus eurem Alltag.

02

Conversation führen

Zwei Nachfragen stellen und prüfen, ob der Kontext hält.

03

Zweite Version

Instructions ändern, beide Versionen mit derselben Frage vergleichen.

Ordner lab-02 · Wer zügig fertig ist: denselben Agenten zusätzlich im Portal öffnen und dort weitertesten

bechtle

Lab 3 — Dem Agenten ein Werkzeug geben.

Variante A

Eigene Function

Eine Funktion als Tool registrieren, den Tool-Call abfangen, ausführen und das Ergebnis zurückgeben. Danach im Log nachvollziehen, wann das Modell sich für den Aufruf entschieden hat.

Variante B

MCP-Server anbinden

Einen vorbereiteten MCP-Server als Tool eintragen und denselben Agenten damit arbeiten lassen. Interessanter, wenn ihr MCP ohnehin auf der Roadmap habt.

Ordner lab-03 · Beide Varianten liegen fertig im Repo — auch die, die ihr nicht wählt

bechtle

Authentifizierung: keine Keys.

Lokal

Euer az-login. Kein Secret in der .env, kein Secret im Repo.

In Azure

Managed Identity der App Service, Function oder Container App. Derselbe Code, andere Herkunft des Tokens.

In der Pipeline

Workload Identity Federation statt hinterlegtem Client Secret.

```
new DefaultAzureCredential()
```

Nie im Frontend

Foundry direkt aus einer Mobile- oder SPA-Anwendung aufzurufen bedeutet, das Credential auszuliefern. Der richtige Weg ist ein Backend-Dienst, der eure eigene API nach außen anbietet.

bechtle

Berechtigungen und ihre Fehlerbilder.

Symptom	Wahrscheinliche Ursache	Erster Blick
401	Gar kein Token — falscher Tenant, abgelaufenes Login	az account show
403	Token da, Rolle fehlt auf Ressource oder Projekt	Zugriffssteuerung im Portal
404 auf Modell	Deployment-Name stimmt nicht oder liegt in einem anderen Projekt	Models + Endpoints
Läuft lokal, nicht in Azure	Managed Identity ohne Rollenzuweisung	Identity-Blade der App

Rollen wirken auf Ressourcen- oder Projektebene und brauchen nach der Zuweisung ein paar Minuten. Wer sofort neu testet, hält eine gültige Konfiguration für kaputt.

bechtle

Tracing: sehen, was der Agent tut.

01

App Insights verbinden

Eine Ressource an das Projekt hängen, Connection String übernehmen.

02

OpenTelemetry einschalten

Vor dem Erzeugen von Client und Agent, sonst fehlen die ersten Spans.

03

Lauf nachvollziehen

Prompt, Tool-Calls, Token und Dauer als eine zusammenhängende Kette.

Ohne Tracing debuggt ihr eine nichtdeterministische Anwendung über console.log. Das trägt genau bis zum ersten Produktivvorfall.

bechtle

Evaluations sind eure Tests.

Eine Prompt-Änderung ist eine Codeänderung ohne Compiler. Was bei klassischer Software die Testsuite leistet, leisten hier Evaluations.

Testdatensatz

20 bis 50 echte Fälle aus eurem Betrieb schlagen 500 erfundene.

Evaluatoren

Mitgelieferte Metriken plus eigene Kriterien, die eure Fachlichkeit abbilden.

In der Pipeline

Läufe sind asynchron: einreichen, auf Abschluss warten, dann bewerten.

bechtle

Guardrails und Content Safety.

Standardmäßig aktiv

Inhaltsfilter laufen mit, ohne dass ihr etwas konfiguriert. Für die meisten Unternehmensszenarien reicht das.

Wo ihr nachsteuert

Fachsprachen mit sensiblem Vokabular — Medizin, Recht, Sicherheit — brauchen angepasste Schwellen, sonst blockt der Filter Legitimes.

Was Filter nicht leisten

Sie prüfen Inhalte, nicht Berechtigungen. Dass ein Agent nur sieht, was der anfragende Nutzer sehen darf, müsst ihr selbst sicherstellen.

Die Berechtigungsfrage ist bei RAG- und SharePoint-Szenarien der kritische Punkt: ein Index kennt eure ACLs nicht von allein.

bechtle

Vier Hebel bei den Kosten.

Kontextlänge

Der mit Abstand größte Posten. Jede Conversation-Runde schleppt die Historie mit — irgendwann muss zusammengefasst oder abgeschnitten werden.

Modell-Mix

Nicht jeder Schritt braucht das größte Modell. Klassifizieren und Routen geht klein, Formulieren groß.

Caching

Stabile System-Instruktionen an den Anfang, damit wiederkehrende Teile günstiger werden.

Abrechnungsmodell

Verbrauchsbasiert für schwankende Last, reservierte Kapazität ab planbarem Dauerbetrieb.

Konkrete Preise bewusst nicht auf der Folie — sie ändern sich schneller als dieser Foliensatz. Die Rechnung für euren Use Case machen wir gemeinsam im Nachgang.

bechtle

Netzwerk und Compliance.

Private Endpoints

Erreichbarkeit nur aus eurem Netz. Dann muss aber auch alles, was den Dienst aufruft, dort stehen — Build-Agents eingeschlossen.

Datenresidenz

Die Region der Ressource entscheidet, welche Modelle verfügbar sind. Verfügbarkeit und Compliance-Anforderung gegeneinander prüfen, bevor ihr euch festlegt.

Verwaltete Tools

Web Search und ähnliche Dienste verlassen euren Perimeter. Das ist eine bewusste Entscheidung, keine technische Nebensache.

Diese drei Punkte gehören in die Architekturentscheidung, nicht in die Umsetzungsphase. Nachträglich abzusichern ist deutlich teurer als von Anfang an einzuplanen.

bechtle

Wie sähe das bei euch aus?

Client
Web, Teams oder Power App

→

Backend-API
Managed Identity, eure Geschäftslogik

→

Foundry Project
Agent-Version, Tools, Conversations

→

Eure Systeme
SharePoint, Ticketsystem, Datenbank

Quer dazu: Entra ID
Wer darf was — vom Client bis zur Datenquelle

Quer dazu: App Insights
Ein Trace über alle Schichten hinweg

?
Wo hört eure Logik auf und fängt der Agent an?

?
Wie kommen Nutzerberechtigungen bis zur Datenquelle durch?

?
Was passiert, wenn der Agent die falsche Antwort gibt?

bechtle

Wie es weitergeht.

Diese Woche

Das Repo läuft weiter gegen euer Projekt. Nehmt einen echten eigenen Fall und baut ihn nach.

Nächster Schritt

Ein Use Case, durchgezogen bis Tracing und Evaluations. Danach wisst ihr, was der Betrieb wirklich kostet.

Wo wir helfen

Architektur-Review, Landing Zone, Begleitung beim ersten Produktivgang.

Material

Workshop-Repo: [Repo-URL]
Foliensatz und Handout: [Link]
SDK-Changelogs als verlässlichste Quelle

Kontakt

Marcel [Nachname]
[E-Mail]
Bechtle [Standort]